

自然语言处理基础（FNLP）课程笔记 v4

AutoPku / Codex

2026-04-28

Contents

说明	4
阅读路线	5
课程知识图谱	6
文件	8
第一章 课程图谱：NLP 任务、概率建模与结构化预测	9
1.1 自然语言处理的基本对象	9
1.2 语言为什么难	9
1.3 概率建模的统一句式	9
1.4 模型学习、解码与评价	10
第二章 词义消歧：从语义歧义到分类问题	11
2.1 WSD 的任务定义	11
2.2 Sense inventory 与 WordNet	11
2.3 三种学习范式	11
2.4 Lesk 与基线	12
2.5 WSD 评价	12
第三章 朴素贝叶斯与生成式分类	13
3.1 从监督学习到概率分类	13
3.2 生成式分类推导	13
3.3 Naive Bayes for WSD	13
3.4 独立性假设的意义	14
第四章 特征表示与 Log-linear 判别式模型	15
4.1 特征：证据的函数化	15
4.2 Bag-of-words 与 TF-IDF	15
4.3 线性打分模型	15
4.4 条件对数似然与梯度	16
4.5 正则化	16
第五章 n-gram 语言模型	17
5.1 语言模型要解决什么	17
5.2 链式法则与 Markov 假设	17
5.3 参数估计	18
5.4 Perplexity	18
5.5 未见事件、插值与平滑	18
第六章 神经语言模型与词表示入口	19
6.1 语言建模作为分类	19
6.2 前馈神经语言模型	19
6.3 CNN、RNN、LSTM、GRU	19

6.4 神经 LM 的训练与评估	19
第七章 模型评估、数据切分与预训练范式	21
7.1 Train/Dev/Test	21
7.2 交叉验证	21
7.3 预训练与微调	21
第八章 分布式语义与词向量	22
8.1 从 WordNet 到分布式语义	22
8.2 共现矩阵与 PPMI	22
8.3 SVD 与低维表示	22
8.4 Word2Vec 与负采样	22
8.5 GloVe 与全局统计	23
第九章 序列标注与 HMM	24
9.1 序列标注任务	24
9.2 POS、tagset 与 BIO	24
9.3 HMM 的生成故事	24
9.4 参数估计	25
9.5 Viterbi 解码	25
9.6 Forward 与 Baum–Welch	25
第十章 神经序列标注、CRF 与 Attention	26
10.1 从 HMM 到 MEMM/CRF	26
10.2 BiLSTM–CRF	26
10.3 Attention 的序列建模意义	27
第十一章 成分句法、PCFG 与 CKY	28
11.1 成分结构	28
11.2 CFG 与 PCFG	28
11.3 CNF 与 CKY	28
11.4 评价	29
第十二章 依存句法	30
12.1 成分与依存的区别	30
12.2 依存关系与图约束	30
12.3 Transition–based parsing	30
12.4 Graph–based parsing	30
12.5 评价	31
第十三章 Seq2Seq、Attention 与 Transformer	32
13.1 Sequence–to–Sequence	32
13.2 Attention for Encoder–Decoder	32
13.3 Transformer	32
13.4 解码策略	33
13.5 BLEU 与生成评价	33
第十四章 全课程公式与算法索引	34
14.1 公式总表	34
14.2 动态规划模板	34
14.3 易混点	34
14.4 最后一轮自测	35
第十五章 含义表示与组合语义：从 FOL 到语义解析 (Lec 9)	36
15.1 为什么需要含义表示	36
15.2 含义表示语言 (MRL) 的五条要求	37
15.3 一阶逻辑 (First–Order Logic, FOL)	37
15.4 事件 Reification 与 Entailment	38

15.5 λ 演算: 让 FOL 可以组合	39
15.6 用 CFG 做组合性语义分析	40
15.7 Quantifier Scope 与组合性的挑战	42
15.8 Semantic Parsing 的两种实现路线	43
15.9 学习一个语义解析器	43
15.10 概念关系	45
15.11 速查	46
第十六章 信息检索: 从布尔模型到稠密检索 (Lec 10)	47
16.1 IR 的任务定义	47
16.2 倒排索引 (Inverted Index)	48
16.3 Boolean Model	49
16.4 Vector Space Model (VSM)	50
16.5 Probabilistic Model: BM25	51
16.6 Pseudo Relevance Feedback (PRF, 补充)	53
16.7 Neural Model: Dense Passage Retrieval (DPR)	53
16.8 评测指标 (补充)	55
16.9 概念关系	56
16.10 速查	56

说明

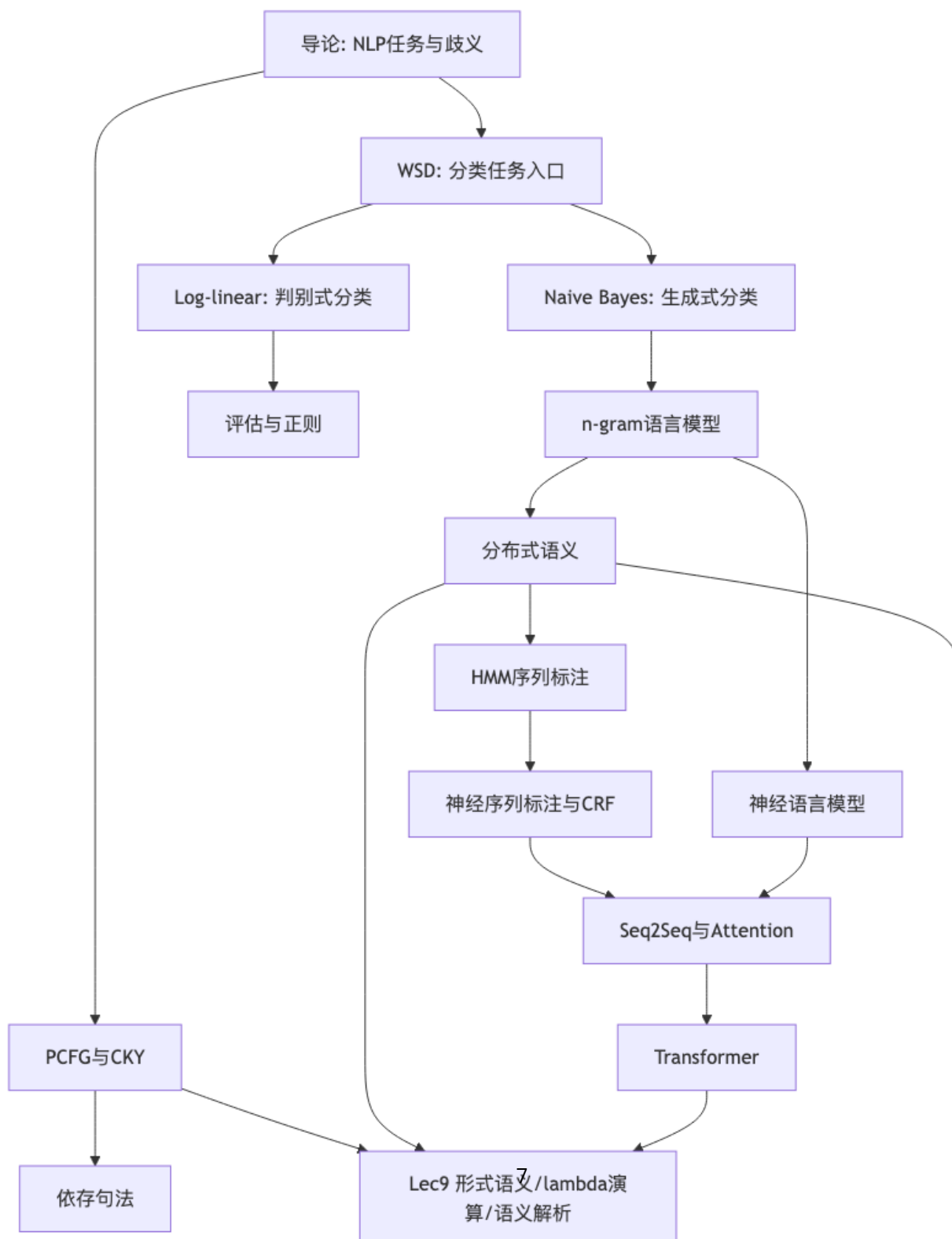
这是重新整理的 v4，不沿用 notes_v2 或 notes_v3 的分讲正文。整理依据是 notes/raw_text/ 中从课件抽取出的文本，覆盖 lec0 至 lec8、lec6_seq 以及 李宏毅--seq2seq_v9.pptx。

本文按课程顺序重写为一份完整讲义：每讲保留课件中的任务动机、模型形式、推导主线、算法模板、评价指标和易错点，删去重复动画页、纯目录页和无助于复习的背景噪声。

阅读路线

目标	建议读法
快速建立全局框架	先读第 1 章课程图谱，再看每讲末尾速查
准备推导题	重点读 Naive Bayes、log-linear、n-gram、HMM、CKY、attention
准备算法题	重点读 Viterbi、Forward、CKY、依存 transition、beam search
准备概念题	对照生成式/判别式、MEMM/CRF、成分/依存、AT/NAT
复习语义与检索	重点读 Lec 9 (FOL + λ 演算 + CFG semantic attachment) 与 Lec 10 (TF-IDF/BM25/DPR)

课程知识图谱



文件

- 主笔记 (Lec 0–8): `fnlp_v4_full.md`
- 新增章节 (Lec 9 含义表示与语义解析): `lec9_sp.md`
- 新增章节 (Lec 10 信息检索): `lec10_ir.md`
- 来源索引: `sources.md`
- PDF: `../pdf/FNLP 课程笔记 _v4.pdf` (截至 Lec 8)、`../pdf/FNLP 课程笔记 _v5.pdf` (含 Lec 9/10)
- 互动 HTML: `index.html`

第一章 课程图谱：NLP 任务、概率建模与结构化预测

[TIP] 学习指南

本章对应 lec0 与 lec1_introduction。核心是建立整门课的任务地图：NLP 不是一组孤立模型，而是在不同输出空间上做预测、打分和搜索。

1.1 自然语言处理的基本对象

自然语言处理 (Natural Language Processing, NLP) 研究怎样让计算系统处理人类语言。课件从课程安排进入主题，但真正的主线很快出现：语言既有词法、句法、语义、语用等层次，也有工程上明确的输入输出。一个 NLP 系统通常不是直接“理解语言”，而是在某个任务定义下把输入 x 映射到输出 y ，并用数据学习这个映射。

课程中的任务可以按输出空间分为四类。第一类是单点分类，例如词义消歧、文本分类；第二类是序列概率建模，例如语言模型；第三类是等长序列标注，例如词性标注、命名实体识别、中文分词；第四类是结构化输出，例如成分句法树、依存图、机器翻译输出序列。输出空间越复杂，模型越需要显式处理全局一致性和搜索。

1.2 语言为什么难

课件导论反复强调歧义。歧义可能出现在词义、词性、短语附着、句法结构、指代、语用意图等层面。一个词可以有多个义项，一句话可以有多种句法树，一段文本的真实意图还可能依赖上下文和世界知识。课程后面引入的分类模型、语言模型、HMM、PCFG、依存句法和 Transformer，本质上都是不同抽象层级上压缩、利用和消解歧义。

另一个困难是组合爆炸。有限词表可以组成无限多句子，有限标签集可以形成指数多标签序列，有限文法规则可以生成大量树结构。暴力枚举通常不可行，因此课程大量使用独立性假设和动态规划：n-gram 用 Markov 假设压缩历史，HMM 用局部转移压缩标签序列，CKY 用 span chart 复用子结构，Transformer 用注意力在可并行结构中建模长距离依赖。

1.3 概率建模的统一句式

整门课可以用三种概率对象串起来。

第一种是条件分布：

$$p(y \mid x).$$

分类、序列标注、句法分析和机器翻译最终都关心给定输入 x 后最可能的输出 y 。判别式模型直接学习这个对象。

第二种是联合分布：

$$p(x, y) = p(y)p(x \mid y).$$

朴素贝叶斯、HMM、PCFG 都先讲一个生成故事，再用 Bayes rule 或最大联合概率做预测。

第三种是序列概率：

$$p(w_{1:n}) = \prod_{i=1}^n p(w_i \mid w_{1:i-1}).$$

语言模型围绕这个对象展开，后续神经语言模型、Seq2Seq decoder 和自回归生成都继承这一形式。

1.4 模型学习、解码与评价

课程中每个模型都要回答三件事。第一，参数如何估计：计数式 MLE、条件对数似然、正则化最大似然、负采样、EM 或神经网络反向传播。第二，预测时如何解码：分类取最大分数，HMM 用 Viterbi，语言模型逐步预测，PCFG 用 CKY，依存句法用 transition 或 graph search，Seq2Seq 用 greedy、beam 或 sampling。第三，怎样评价：分类看 accuracy/F1，语言模型看 perplexity，句法看 PARSEVAL/UAS/LAS，生成看 BLEU 或任务指标。

[WARN] 易错点

模型名字不是复习的中心。先问输出是什么，再问概率对象是什么，最后问训练目标和解码算法。这样遇到新模型也能放回课程框架里。

第二章 词义消歧：从语义歧义到分类问题

[TIP] 学习指南

本章对应 lec2_wsd。它是课程进入机器学习模型的入口：给定目标词及上下文，从固定义项清单中选出正确义项。

2.1 WSD 的任务定义

词义消歧 (Word Sense Disambiguation, WSD) 的输入是目标词 w 、上下文 C 和义项集合 $S = \{s_1, \dots, s_K\}$ ，输出是目标词在该上下文中的义项：

$$s^* = \arg \max_{s \in S} \text{Score}(s, C).$$

课件从“words may have multiple distinct meanings”出发，区分同形异义和一词多义。英语 **bank** 可以指金融机构，也可以指河岸；汉语中“长”在“长高”和“很长”里读音和意义都不同。WSD 不是只为词典服务，它直接影响语音合成、机器翻译、信息抽取、问答和实体消歧。

2.2 Sense inventory 与 WordNet

WSD 必须先规定候选义项。WordNet 是课件重点介绍的英文义项资源：它以 synset 组织名词、动词、形容词和副词的词义，并给出 gloss、例句、同义词、上位词、下位词等关系。对 WSD 来说，WordNet 的作用是定义标签空间；对 Lesk 这类知识库方法来说，它还提供了可比较的释义文本。

义项清单也带来困难。清单越细，模型越难预测，标注一致性越差；清单越粗，又可能不能满足真实任务需求。因此 WSD 既是机器学习问题，也是任务定义问题。

2.3 三种学习范式

监督式 WSD 把问题看成多分类：训练集包含 $(x^{(i)}, y^{(i)})$ ，其中 $x^{(i)}$ 是含目标词的上下文， $y^{(i)}$ 是义项标签。模型从上下文抽取特征，学习 $g(x) = y$ 或 $p(y | x)$ 。

半监督式 WSD 使用少量种子或高置信预测扩充标注。课件中的 Yarowsky 思路依赖两个经验规律：one sense per discourse 与 one sense per collocation。同一文档中的同一个词倾向共享义项，固定搭配也倾向强烈指向一个义项。

无监督式 WSD 更准确地说是词义归纳 (Word Sense Induction, WSI)：模型从上下文相似性中聚类出不同用法，但聚类结果还需要映射回人工义项清单。

2.4 Lesk 与基线

Most Frequent Sense (MFS) 是 WSD 的强基线：永远选训练语料中最常见的义项。它简单但很难被弱模型超过，因为自然语言中的义项分布通常长尾且偏斜。

Lesk 算法不依赖标注语料。它比较上下文词与候选义项 gloss 的重叠：

$$s^* = \arg \max_{s \in S} |G(s) \cap C|.$$

扩展版 Lesk 会加入上位词、下位词 gloss，或给词重叠加权。Lesk 的核心假设是：正确义项的释义词更可能与上下文词共享语义线索。

2.5 WSD 评价

WSD 常用 accuracy、precision、recall、F1。若是 all-words setting，模型需要对文本中所有开放类词逐一消歧；若是 lexical-sample setting，只对预先指定的目标词消歧。多类 F1 可以 macro-average，也可以 micro-average。macro 更关注小类，micro 更受大类影响。

[TIP] 本讲速查

WSD = 固定义项清单 + 上下文特征 + 分类/匹配。WordNet 给标签空间，MFS 给强基线，Lesk 用 gloss 重叠，监督方法用标注数据学习 $p(s \mid C)$ 或打分函数。

第三章 朴素贝叶斯与生成式分类

[TIP] 学习指南

本章对应 lec3_classification。它承接 WSD，说明如何从 $p(y | x)$ 目标转向生成式建模 $p(x, y)$ 。

3.1 从监督学习到概率分类

监督学习给定训练样本：

$$(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)}),$$

目标是学习函数 $g : x \mapsto y$ 。概率视角下，分类器选择：

$$g(x) = \arg \max_y p(y | x).$$

课件提出两个视角：判别式模型直接学习 $p(y | x)$ ；生成式模型先学习 $p(x, y)$ ，再由 Bayes rule 推出分类规则。

3.2 生成式分类推导

Bayes rule 给出：

$$p(y | x) = \frac{p(x, y)}{p(x)} = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}.$$

对固定 x ，分母与候选标签无关，因此：

$$\arg \max_y p(y | x) = \arg \max_y p(x | y)p(y) = \arg \max_y p(x, y).$$

这就是生成式分类器的通用形式。模型讲一个“先产生标签 y ，再根据标签产生观察 x ”的故事，然后估计先验 $p(y)$ 和似然 $p(x | y)$ 。

3.3 Naive Bayes for WSD

在 WSD 中， y 是义项 s ， x 是上下文 C 。朴素贝叶斯的目标是：

$$s^* = \arg \max_{s_k} P(C | s_k)P(s_k).$$

如果上下文由特征词 v_x 构成，Naive Bayes 作条件独立假设：

$$P(C \mid s_k) = \prod_{v_x \in C} P(v_x \mid s_k).$$

最终：

$$s^* = \arg \max_{s_k} P(s_k) \prod_{v_x \in C} P(v_x \mid s_k)$$

参数由计数估计：

$$P(v_x \mid s_k) = \frac{\text{Count}(v_x, s_k)}{\sum_{v \in V} \text{Count}(v, s_k)}, \quad P(s_k) = \frac{\text{Count}(s_k)}{\text{Count}(w)}.$$

3.4 独立性假设的意义

条件独立假设在语言中明显不成立。词与词之间会共现，语法位置和语义场也会相关。但该假设让参数数量从难以估计的联合上下文分布降为每个特征在每个类别下的条件概率。课程中“THIS IS NOT TRUE! but we then have an easier model”这句话就是本章重点：好的概率模型不总是完全真实，而是要在可估计性和表达能力之间折中。

实际实现时通常使用 log space：

$$s^* = \arg \max_{s_k} \left[\log P(s_k) + \sum_{v_x \in C} \log P(v_x \mid s_k) \right],$$

避免大量小概率相乘造成数值下溢。

[TIP] 本讲速查

Naive Bayes 是生成式模型。推导顺序是 $p(y \mid x)$ 到 $p(x \mid y)p(y)$ ，再用条件独立把 $p(x \mid y)$ 拆成特征似然乘积。

第四章 特征表示与 Log-linear 判别式模型

[TIP] 学习指南

本章对应 lec3_loglinear。课件从 WSD 的“线索”出发，转向特征函数、线性打分、条件概率归一化和正则化。

4.1 特征：证据的函数化

课件把 feature 定义为描述观察数据 x 与候选标签 y 之间某些方面的证据。一个特征通常写为：

$$f_i(x, y) \in \mathbb{R}.$$

在 NLP 中它常是二值 indicator。例如对 bank 做 WSD：

$$f_i(x, y) = \begin{cases} 1, & \text{if previous word is transfer and } y = \text{FINANCIAL}, \\ 0, & \text{otherwise.} \end{cases}$$

特征可以来自目标词、前后词、POS、前后缀、大小写、句法关系，也可以是 bag-of-words、TF-IDF、dense embedding。课程中新闻分类例子说明，同一篇文本在不同候选标签下会激活不同的 label-specific word features。

4.2 Bag-of-words 与 TF-IDF

Bag-of-words 把文档表示为词表维度上的向量，忽略顺序。它可以是 0/1 one-hot 出现向量，也可以是词频向量，还可以用 TF-IDF 加权：

$$\text{TF}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d), & \text{count}(t, d) > 0, \\ 0, & \text{otherwise,} \end{cases}$$
$$\text{IDF}_t = \log_{10} \frac{N}{\text{DF}_t}, \quad \text{TFIDF}_{t,d} = \text{TF}_{t,d} \text{IDF}_t.$$

TF-IDF 的直觉是：在当前文档中频繁出现的词重要，但在所有文档中都常见的词要降权。它是许多线性文本分类模型的基础输入。

4.3 线性打分模型

最简单的特征判别式模型对每个候选标签打分：

$$\text{score}(x, y) = \sum_i \lambda_i f_i(x, y) = \lambda^\top f(x, y).$$

预测：

$$y^* = \arg \max_y \text{score}(x, y).$$

这已经能做分类，但它还不是概率模型。log-linear 模型把线性分数指数化并归一化：

$$p_{\theta}(y | x) = \frac{\exp(\theta^{\top} f(x, y))}{Z_{\theta}(x)}$$

其中：

$$Z_{\theta}(x) = \sum_{y'} \exp(\theta^{\top} f(x, y')).$$

4.4 条件对数似然与梯度

单个样本的条件对数似然：

$$\ell(\theta) = \log p_{\theta}(y | x) = \theta^{\top} f(x, y) - \log Z_{\theta}(x).$$

对参数 θ_k 求导：

$$\frac{\partial \ell}{\partial \theta_k} = f_k(x, y) - \sum_{y'} p_{\theta}(y' | x) f_k(x, y').$$

全数据上就是：

$$\nabla_{\theta} L = \text{empirical feature count} - \text{expected feature count}$$

这说明训练的目标是让模型期望特征计数靠近训练数据中的经验特征计数。最大熵模型的 moment matching 直觉就来自这里。

4.5 正则化

如果某个特征和某个标签在训练集中完全共现，未正则化模型可能把对应权重推向无穷大。L2 正则则在目标函数中加入：

$$L_{\text{reg}}(\theta) = L(\theta) - \frac{\alpha}{2} \|\theta\|^2,$$

梯度变为：

$$\nabla L_{\text{reg}} = \nabla L - \alpha \theta.$$

正则化的作用是限制参数规模，降低过拟合，并让权重在证据不足时回到较小值。

[TIP] 本讲速查

Log-linear = 特征函数 + 线性分数 + 指数化 + 条件归一化。最重要推导是 $\nabla = \text{empirical} - \text{expected}$ ，最重要工程问题是正则化。

第五章 n-gram 语言模型

[TIP] 学习指南

本章对应 lec4_lm。语言模型给词序列分配概率，是后续神经 LM 和生成模型的基础。

5.1 语言模型要解决什么

课件用“the dog barks”、“the dog laughs”、“dog the bark a laugh smile”引出语言模型：给定一个词串，判断它像不像某种语言中的合法表达。形式化地，语言模型是在所有词序列 S 上定义概率分布：

$$\sum_{s \in S} p(s) = 1, \quad p(s) \geq 0.$$

好句子应获得较大概率，坏句子应获得较小概率。

语言模型在语音识别、OCR、手写识别、中文分词、机器翻译、语言生成、问答和对话中都重要。课件中的 recognize speech / wreck a nice beach 例子说明，声学或视觉输入可能给出多个候选，语言模型负责选择更像自然语言的那个。

5.2 链式法则与 Markov 假设

对句子 $w_{1:n}$ ：

$$p(w_{1:n}) = \prod_{i=1}^n p(w_i | w_{1:i-1}).$$

完整历史太长，无法可靠估计。n-gram 作 Markov 近似：

$$p(w_i | w_{1:i-1}) \approx p(w_i | w_{i-n+1:i-1}).$$

常见模型：

模型	假设	形式
unigram	不看历史	$p(w_i)$
bigram	看前 1 个词	$p(w_i w_{i-1})$
trigram	看前 2 个词	$p(w_i w_{i-2}, w_{i-1})$

trigram 句子概率：

$$p(w_{1:n}) = p(w_1)p(w_2 | w_1) \prod_{i=3}^n p(w_i | w_{i-2}, w_{i-1}).$$

5.3 参数估计

MLE 用计数比：

$$\hat{p}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})},$$
$$\hat{p}(w_i | w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}.$$

课件用 Gigaword 规模说明参数空间极大且稀疏。即使 10 million sentences、275 million words，很多可能 n-gram 也从未出现。MLE 对未见事件给零概率，这会让包含未见 n-gram 的测试句概率变成零。

5.4 Perplexity

语言模型在测试集 D 上的目标是给真实句子高概率。交叉熵：

$$H(D) = -\frac{1}{M} \sum_{s \in D} \log_2 p(s),$$

其中 M 是测试集 token 数。困惑度：

$$\text{PPL}(D) = 2^{H(D)} = 2^{-\frac{1}{M} \sum_{s \in D} \log_2 p(s)}$$

PPL 越低，模型越好。直觉上，perplexity 是模型平均每步面对的有效候选数。

5.5 未见事件、插值与平滑

为避免零概率，需要把概率质量分给未见事件。线性插值把不同阶模型混合：

$$p(w_i | w_{i-2}, w_{i-1}) = \lambda_3 p_{\text{tri}} + \lambda_2 p_{\text{bi}} + \lambda_1 p_{\text{uni}},$$

其中 $\lambda_1 + \lambda_2 + \lambda_3 = 1$ 。

常见平滑包括 add-one、Good-Turing、absolute discounting、Kneser-Ney。Add-one 简单但常过度平滑；Good-Turing 用低频事件估计未见质量；Kneser-Ney 的关键是 continuation probability：一个词作为新上下文延续的能力，比它的总频数更能解释某些 bigram 概率。

[TIP] 本讲速查

n-gram LM = 链式法则 + Markov 假设 + 计数估计 + 平滑。考试最常见是算 trigram 概率、PPL，解释数据稀疏和平滑动机。

第六章 神经语言模型与词表示入口

[TIP] 学习指南

本章对应 lec4_nlm。课件把语言建模重新看成分类问题：给定上下文，预测下一个词；神经网络的关键收益是用 dense representation 缓解稀疏。

6.1 语言建模作为分类

神经语言模型把每个位置的预测写为：

$$p(w_t | w_{<t}) = \text{softmax}(g(h_t)),$$

其中 h_t 是上下文表示，输出维度是词表大小。相比 n-gram 直接查表，神经 LM 学习共享参数：相似上下文和相似词可以通过向量空间共享统计强度。

6.2 前馈神经语言模型

早期 NNLM 使用固定窗口。例如给定 $w_{t-k}, \dots, w_{t-1}$ ，先查词向量，再拼接或平均，经过非线性层预测 w_t ：

$$h = \sigma(W[e(w_{t-k}); \dots; e(w_{t-1})] + b),$$

$$p(w_t | h) = \text{softmax}(Uh + c).$$

它比 n-gram 好的地方是：参数不再为每个离散 history 独立存储，而是通过 embedding 和网络权重共享。

6.3 CNN、RNN、LSTM、GRU

CNN 可以用卷积窗口抽取局部 n-gram-like 模式。RNN 用递归状态表示任意长度历史：

$$h_t = \phi(W_x e(w_t) + W_h h_{t-1} + b).$$

普通 RNN 面临梯度消失/爆炸。LSTM 和 GRU 通过门控机制控制记忆写入、遗忘和输出，使模型更适合长距离依赖。

6.4 神经 LM 的训练与评估

训练目标通常是最大化语料 log-likelihood：

$$\sum_t \log p_\theta(w_t | w_{<t}).$$

评估仍使用 perplexity。注意：PPL 是语言模型内在指标，不一定完全等价于下游任务性能；但在同一数据和词表设置下，低 PPL 通常表示更好的预测能力。

[TIP] 本讲速查

神经 LM 的核心变化是从稀疏 history table 转为 dense context representation。它仍在做下一个词分类，仍用 softmax 概率和 PPL 评价。

第七章 模型评估、数据切分与预训练范式

[TIP] 学习指南

本章对应 lec4_pre。课件文本抽取中噪声较多，但核心主题是训练、验证、测试以及预训练/微调的实验协议。

7.1 Train/Dev/Test

机器学习实验必须区分训练集、验证集和测试集。训练集用于拟合参数；验证集用于选择超参数、特征、模型结构和早停点；测试集只用于最终报告。若频繁根据测试集调整模型，就会把测试集泄漏为开发集，得到过于乐观的估计。

7.2 交叉验证

当数据较少，可以使用 K 折交叉验证：把数据分成 K 份，每次用一份作验证，其余作训练，重复 K 次后平均结果。它能降低单次划分带来的方差，但训练成本更高。

7.3 预训练与微调

预训练 (pre-training) 先在大规模通用语料上学习表示或语言模型，再在下游任务上微调 (fine-tuning)。从课程主线看，它延续了神经语言模型：语言模型训练提供可迁移的词或上下文表示，下游任务只需少量标注即可利用这些表示。

[WARN] 易错点

不能用测试集调参。dev set 的作用是替代测试集参与模型选择；test set 的作用是估计最终泛化性能。

第八章 分布式语义与词向量

[TIP] 学习指南

本章对应 lec5_dist。它从 WordNet 等符号语义资源转向 distributional hypothesis：相似上下文中的词倾向意义相似。

8.1 从 WordNet 到分布式语义

WordNet 用人工语义关系组织词义，优点是结构清晰，缺点是覆盖、更新和粒度都受人工维护限制。分布式语义从语料中学习词义表示，核心假设是：

words that appear in similar contexts tend to have similar meanings.

因此，词的意义可以由它出现的上下文分布来近似。

8.2 共现矩阵与 PPMI

构造词-上下文矩阵 X ，其中 X_{wc} 表示词 w 与上下文 c 的共现次数。PMI 衡量二者共现是否超过独立假设：

$$\text{PMI}(w, c) = \log \frac{p(w, c)}{p(w)p(c)}.$$

PPMI 截断负值：

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0).$$

PPMI 比 raw count 更关注有信息量的搭配，但矩阵仍然高维稀疏。

8.3 SVD 与低维表示

对共现矩阵做低秩分解：

$$X \approx U_k \Sigma_k V_k^\top.$$

词向量可取 $U_k \Sigma_k$ 或相关变体。SVD 的意义是把高维稀疏共现压缩成低维 dense representation，保留主要统计结构。

8.4 Word2Vec 与负采样

Word2Vec 包含 CBOW 和 Skip-gram。CBOW 用上下文预测中心词；Skip-gram 用中心词预测上下文。Skip-gram 目标可写为最大化：

$$\sum_{(w,c) \in D} \log p(c | w).$$

完整 softmax 成本高，负采样把问题转化为二分类：真实 (w, c) 对应正例，随机采样 (w, c') 对应负例：

$$\log \sigma(v_c^\top v_w) + \sum_{j=1}^k \mathbb{E}_{c_j \sim P_n} \log \sigma(-v_{c_j}^\top v_w).$$

8.5 GloVe 与全局统计

GloVe 拟合词向量内积与共现计数的对数之间的关系：

$$J = \sum_{i,j} f(X_{ij})(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2.$$

它位于矩阵分解与局部预测之间：既利用全局共现矩阵，又学习 dense word vectors。

[TIP] 本讲速查

分布式语义从“词典关系”转向“上下文相似”。PPMI/SVD 是计数矩阵路线，Word2Vec 是预测路线，GloVe 是全局共现拟合路线。

第九章 序列标注与 HMM

[TIP] 学习指南

本章对应 lec6_hmm。核心是把 POS tagging、NER、分词等任务统一为输入序列到标签序列的预测，并用 HMM 建模联合概率。

9.1 序列标注任务

序列标注给定 $x_{1:n}$ ，输出等长标签序列 $y_{1:n}$ 。典型任务包括 POS tagging、NER、chunking、中文分词、case restoration、accent restoration。与单点分类不同，标签之间存在依赖。例如 POS tag 的合法转移、BIO 标签的边界约束，都不能用独立分类完全表达。

9.2 POS、tagset 与 BIO

POS 描述词的句法类别。闭类词如 determiners、pronouns、prepositions，成员较固定；开类词如 nouns、verbs、adjectives、adverbs，成员可增长。tagset 是对这些类别的标准化编码，例如 Penn Treebank tagset 和 Universal POS。

BIO 把 span 识别转成 token-level labeling：B 表示开始，I 表示内部，O 表示外部。带类型时可写成 B-ORG、I-ORG、B-LOC 等。

9.3 HMM 的生成故事

HMM 是生成式序列模型，包含隐状态 y_t 和观察 x_t 。bigram HMM 参数：

参数	含义
$\pi(y)$	初始状态分布
$a_{ij} = p(y_t = j \mid y_{t-1} = i)$	转移概率
$b_j(x) = p(x_t = x \mid y_t = j)$	发射概率

联合概率：

$$p(x_{1:n}, y_{1:n}) = \pi(y_1) b_{y_1}(x_1) \prod_{t=2}^n a_{y_{t-1}, y_t} b_{y_t}(x_t).$$

trigram HMM 用 $p(y_t \mid y_{t-2}, y_{t-1})$ ，因此 Viterbi 状态要保留两个最近标签。

9.4 参数估计

有标签训练集上, HMM 参数用计数式 MLE:

$$\hat{p}(y_t | y_{t-1}) = \frac{c(y_{t-1}, y_t)}{c(y_{t-1})},$$

$$\hat{p}(x_t | y_t) = \frac{c(y_t, x_t)}{c(y_t)}.$$

这与 n-gram LM 的估计形式完全同构。实际需要平滑, 并对 OOV 词设置 <UNK> 或形态特征回退。

9.5 Viterbi 解码

目标:

$$\hat{y}_{1:n} = \arg \max_{y_{1:n}} p(x_{1:n}, y_{1:n}).$$

bigram Viterbi 定义:

$$\delta_t(j) = \max_{y_{1:t-1}} p(y_{1:t-1}, y_t = j, x_{1:t}).$$

递推:

$$\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(x_t)$$

回指针:

$$\psi_t(j) = \arg \max_i \delta_{t-1}(i) a_{ij}.$$

复杂度 $O(n|S|^2)$ 。

trigram Viterbi:

$$\pi(k, u, v) = \max_w \pi(k-1, w, u) p(v | w, u) p(x_k | v),$$

复杂度 $O(n|S|^3)$ 。

9.6 Forward 与 Baum-Welch

Forward 算法计算观察序列概率:

$$\alpha_t(j) = p(x_{1:t}, y_t = j).$$

递推:

$$\alpha_t(j) = \left[\sum_i \alpha_{t-1}(i) a_{ij} \right] b_j(x_t).$$

它与 Viterbi 的差别是 sum 与 max。无标签训练时, Baum-Welch 使用 EM 估计隐状态后验和参数。

[TIP] 本讲速查

HMM 三问: likelihood 用 Forward, decoding 用 Viterbi, learning 有标签用 MLE、无标签用 Baum-Welch。Viterbi 是 max, Forward 是 sum。

第十章 神经序列标注、CRF 与 Attention

[TIP] 学习指南

本章综合 lec6_seq 中序列模型部分。它把 HMM 的离散生成式序列建模推进到神经编码、CRF 全局归一和 attention。

10.1 从 HMM 到 MEMM/CRF

HMM 的局限是观察模型过弱： x_t 只依赖 y_t ，难以使用整句上下文特征。MEMM 改成每一步条件模型：

$$p(y_t | y_{t-1}, x) = \frac{\exp(\theta^\top f(y_{t-1}, y_t, x, t))}{\sum_{y'} \exp(\theta^\top f(y_{t-1}, y', x, t))}.$$

它能使用丰富特征，但局部归一导致 label bias。

Linear-chain CRF 对整条路径全局归一：

$$p(y_{1:n} | x) = \frac{\exp(\sum_t \theta^\top f(y_{t-1}, y_t, x, t))}{\sum_{y'_{1:n}} \exp(\sum_t \theta^\top f(y'_{t-1}, y'_t, x, t))}.$$

CRF 解码仍可用 Viterbi，训练中的归一化项用 Forward 类动态规划计算。

10.2 BiLSTM-CRF

神经序列标注常用 BiLSTM 编码输入：

$$h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

每个位置得到 emission score，再加上标签转移分数：

$$\text{Score}(x, y) = \sum_t \text{emit}(h_t, y_t) + \sum_t T_{y_{t-1}, y_t}.$$

CRF 层用全局 softmax：

$$p(y | x) = \frac{\exp(\text{Score}(x, y))}{\sum_{y'} \exp(\text{Score}(x, y'))}.$$

这样既利用神经网络的上下文表示，又保留标签转移一致性。

10.3 Attention 的序列建模意义

Attention 让模型在当前决策时选择性读取其他位置。对 query q 、keys k_i 、values v_i ：

$$\alpha_i = \text{softmax}(q^\top k_i), \quad c = \sum_i \alpha_i v_i.$$

它解决了固定长度状态难以承载全部历史的问题，也为后续 Transformer 铺路。

[TIP] 本讲速查

HMM 用生成式局部假设，MEMM 用局部条件概率，CRF 用全局条件概率。BiLSTM-CRF 是神经 emission + CRF transition。

第十一章 成分句法、PCFG 与 CKY

[TIP] 学习指南

本章对应 lec7_phrase。核心是从词序列输出短语结构树，并用 PCFG 给树打概率，用 CKY 做动态规划解析。

11.1 成分结构

成分句法 (constituency parsing) 把句子分析成层级短语结构，如 NP、VP、PP。短语结构树强调一个片段如何作为整体在更大结构中发挥作用。它与后面的依存句法不同：成分句法关注 span 和短语类别，依存句法关注词与词之间的 head-dependent 关系。

11.2 CFG 与 PCFG

上下文无关文法 (CFG) 包含非终结符、终结符、规则和起始符号。规则形如：

$$A \rightarrow \beta.$$

PCFG 给每条规则概率：

$$\sum_{\beta} P(A \rightarrow \beta) = 1.$$

一棵树的概率是其中所有规则概率的乘积：

$$P(T) = \prod_{r \in T} P(r).$$

参数 MLE：

$$\hat{P}(A \rightarrow \beta) = \frac{c(A \rightarrow \beta)}{\sum_{\beta'} c(A \rightarrow \beta')}.$$

11.3 CNF 与 CKY

CKY 通常要求文法为 Chomsky Normal Form：规则为 $A \rightarrow BC$ 或 $A \rightarrow w$ 。定义 chart：

$$\pi[i, j, A] = \text{span}(i, j) \text{ 被 } A \text{ 推导的最高概率}.$$

初始化：

$$\pi[i, i+1, A] = P(A \rightarrow w_i).$$

递推：

$$\pi[i, j, A] = \max_{A \rightarrow BC} \max_{i < k < j} P(A \rightarrow BC) \pi[i, k, B] \pi[k, j, C]$$

答案在 $\pi[0, n, S]$ ，回指针记录最佳规则和切分点。

11.4 评价

成分句法常用 PARSEVAL：比较预测树和 gold tree 的 labeled constituents，计算 precision、recall、F1。句法树的边界和标签都重要，因此只预测出大致结构还不够。

[TIP] 本讲速查

PCFG = CFG 规则概率化；树概率 = 规则概率乘积；CKY = span chart + split point + grammar rule。

第十二章 依存句法

[TIP] 学习指南

本章对应 lec8_dp。依存句法不输出短语树，而输出以词为节点的有根依存图。

12.1 成分与依存的区别

成分句法关心 span 组成的短语层级；依存句法关心词与词之间的二元关系。一个依存弧 (h, d, l) 表示 head h 支配 dependent d ，关系标签为 l 。整个句子的依存结构通常是一棵以 ROOT 为根的有向树。

12.2 依存关系与图约束

依存图通常要求：

1. 每个非 ROOT 词恰好有一个 head。
2. ROOT 没有 head。
3. 图连通且无环。
4. 某些体系要求投射性 (projectivity)。

投射性可理解为：把词按句子顺序写在一行，依存弧画在上方时不交叉。非投射结构在自由语序语言中更常见。

12.3 Transition-based parsing

Transition-based parser 维护 stack、buffer 和 arcs，通过动作逐步构造树。arc-standard 系统常见动作：

动作	作用
SHIFT	buffer 首词移入 stack
LEFT-ARC	建立右侧词指向左侧词的弧，并弹出 dependent
RIGHT-ARC	建立左侧词指向右侧词的弧，并弹出 dependent

分类器根据当前配置选择下一动作。优点是速度快，常可线性时间解析；缺点是早期错误会传播。

12.4 Graph-based parsing

Graph-based parser 给所有可能弧打分，然后寻找最高分合法依存树：

$$T^* = \arg \max_T \sum_{(h,d) \in T} s(h, d).$$

非投射解析可用最大生成树算法，如 Chu-Liu/Edmonds；投射解析可用 Eisner 动态规划。

12.5 评价

依存句法常用：

- UAS (Unlabeled Attachment Score): head 是否正确。
- LAS (Labeled Attachment Score): head 和 dependency label 是否都正确。

[TIP] 本讲速查

依存句法输出词级有根树。Transition-based 是动作序列搜索，graph-based 是全局树搜索；UAS 看 head，LAS 同时看 head 和 label。

第十三章 Seq2Seq、Attention 与 Transformer

[TIP] 学习指南

本章对应 lec6_seq 与 李宏毅--seq2seq_v9.pptx。核心是从等长标签序列推广到输入输出长度不同的序列生成。

13.1 Sequence-to-Sequence

Seq2Seq 任务输入 $x_{1:S}$ ，输出 $y_{1:T}$ ，两者长度可以不同。典型任务是机器翻译、摘要、对话、语音识别、纠错等。Encoder 把输入编码为表示，Decoder 自回归生成输出：

$$p(y_{1:T} | x_{1:S}) = \prod_{t=1}^T p(y_t | y_{<t}, x_{1:S}).$$

训练常用 teacher forcing：第 t 步 decoder 输入 gold prefix。推理时只能输入自己已生成的 token，因此会产生 exposure bias。

13.2 Attention for Encoder-Decoder

早期 Seq2Seq 把整个输入压进一个固定向量，长句效果差。Attention 让 decoder 每一步动态读取 encoder hidden states：

$$\begin{aligned} e_{t,s} &= \text{score}(s_{t-1}, h_s), \\ \alpha_{t,s} &= \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \\ c_t &= \sum_s \alpha_{t,s} h_s. \end{aligned}$$

然后用 c_t 和 decoder state 预测 y_t 。

13.3 Transformer

Transformer 抛弃 RNN recurrence，使用 self-attention 和 feed-forward network 堆叠。Scaled dot-product attention：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

除以 $\sqrt{d_k}$ 是为了避免维度变大后点积方差过大导致 softmax 饱和。

Multi-head attention:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V),$$
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O.$$

Encoder block: self-attention、residual、layer norm、feed-forward。Decoder block 额外包含 masked self-attention 和 cross-attention。因为 self-attention 本身不编码顺序，模型需要 positional encoding。

13.4 解码策略

Greedy decoding 每步选概率最大 token，速度快但容易错过全局更优序列。Beam search 保留 B 个高分前缀：

$$\text{score}(y_{1:t}) = \sum_{i=1}^t \log p(y_i \mid y_{<i}, x).$$

开放式生成中还会使用 sampling、top-k、top-p 等策略。课程中强调：任务目标不同，解码策略也不同；翻译等答案较确定任务常用 beam，开放生成不宜只追求最高概率。

13.5 BLEU 与生成评价

BLEU 用 n-gram precision 和 brevity penalty 评价机器翻译：

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right).$$

它便于自动评估，但不能完全衡量语义正确性、事实性和可读性。

[TIP] 本讲速查

Seq2Seq = encoder + autoregressive decoder; attention = decoder 每步读输入; Transformer = self-attention + multi-head + FFN + residual/layer norm + positional encoding。

第十四章 全课程公式与算法索引

14.1 公式总表

模块	核心公式
WSD/NB	$s^* = \arg \max_s P(s) \prod_{v \in C} P(v s)$
Generative classification	$\arg \max_y p(y x) = \arg \max_y p(x y)p(y)$
Log-linear	$p(y x) = \exp(\theta^\top f(x, y)) / Z(x)$
Log-linear gradient	$\nabla = \text{empirical} - \text{expected}$
n-gram	$p(w_{1:n}) \approx \prod_i p(w_i w_{i-n+1:i-1})$
PPL	$\text{PPL} = 2^{-\frac{1}{M} \sum \log_2 p(s)}$
HMM	$p(x, y) = \pi(y_1) b_{y_1}(x_1) \prod_t a_{y_{t-1}, y_t} b_{y_t}(x_t)$
Viterbi	$\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(x_t)$
CRF	$p(y x) = \exp(\text{Score}(x, y)) / \sum_{y'} \exp(\text{Score}(x, y'))$
CKY	$\pi[i, j, A] = \max_{A \rightarrow BC, k} P(A \rightarrow BC) \pi[i, k, B] \pi[k, j, C]$
Attention	$\text{softmax}(QK^\top / \sqrt{d_k})V$

14.2 动态规划模板

Viterbi: 状态是“位置 + 当前标签”，递推取 max，回指针恢复最优路径。

Forward: 状态是“位置 + 当前标签”，递推取 sum，终止时求总 likelihood。

CKY: 状态是“span + nonterminal”，递推枚举 split point 和规则，回指针恢复最优树。

Eisner: 状态是“span + 方向 + complete/incomplete”，用于投射依存树。

14.3 易混点

易混概念	区分
生成式 vs 判别式	生成式建 $p(x, y)$; 判别式建 $p(y x)$
Viterbi vs Forward	Viterbi 求最优路径 max; Forward 求总概率 sum
MEMM vs CRF	MEMM 局部归一; CRF 全局归一
成分 vs 依存	成分看短语 span; 依存看 head-dependent
PPL vs Accuracy	PPL 评价概率模型; Accuracy 评价离散预测
Beam vs Sampling	Beam 追求高概率序列; sampling 保留多样性

14.4 最后一轮自测

1. 能否从 Bayes rule 推出 Naive Bayes 的 WSD 决策式?
2. 能否写出 log-linear 的 $Z(x)$, 并推导梯度为什么是 empirical minus expected?
3. 能否手算 trigram 句子概率和 perplexity?
4. 能否给定 HMM 参数填 Viterbi 表, 并解释与 Forward 的差别?
5. 能否把 PCFG 转成 CKY chart 的初始化、递推和终止?
6. 能否解释 label bias, 并说明 CRF 如何通过全局归一缓解?
7. 能否写出 scaled dot-product attention, 并说明 multi-head 和 positional encoding 的必要性?

[TIP] 复习结论

整门课的核心不是背模型名, 而是建立“任务输出空间决定概率对象, 概率对象决定训练目标, 结构假设决定解码算法, 评价指标反映任务目标”的链条。

第十五章 含义表示与组合语义：从 FOL 到语义解析 (Lec 9)

[TIP] 学习指南

本章核心：把自然语言句子翻译成结构化、可验证、可推理的形式语义表示 (Meaning Representation, MR)；并按“组合性原则”用 CFG + λ 演算自底向上构建该表示。前置：第 1 章概率建模框架；第 5 章分布式语义 (vector space) 提供对比；第 11–12 章 PCFG / CKY 提供句法骨架。复习关注点：1. 形式语义对 MRL 的五个要求；2. FOL 的语法成分 (term/predicate/quantifier/connective) 与 λ 演算；3. 事件 reification 与 entailment；4. CFG + semantic attachment 计算 S.sem，能手算“AyCaramba serves Meat”与“Every kid cries”；5. 类型提升 (type raising) 解决 quantifier scope；6. semantic parsing 的两种实现路线 (integrate vs pipeline)，及它们与现代神经/LLM Text-to-SQL 的关系。

15.1 为什么需要含义表示

15.1.1 分布式语义的不足

前面的章节用向量空间模型 (Vector Space Model, VSM) 给词、句子学到 dense embedding。它在下游分类、检索任务上很有用，但有两个本质局限：

1. **组合性 (compositionality)**：已有 $v_{\text{apple}}, v_{\text{red}}, v_{\text{yellow}}$ ，怎样表示 red apple 与 yellow apple？简单相加或拼接并不能保证逻辑一致性。
2. **推理 (inference)**：从“Tom likes fruit”+ “apple is a kind of fruit”推出“Tom likes apple”，需要符号化、可演绎的表示。

[NOTE] 直觉

向量是连续、隐式的语义近似；FOL 是离散、显式的语义骨架。课程后半段把这两条线缝合起来——LLM 学了 vector，但要做 verifiable answering 又回到 SQL / SPARQL 这类 formal MR。

15.1.2 一个标杆例子

问：Did Poland reduce its carbon emissions since 1989? 文档：Due to the collapse of the industrial sector after the end of communism in 1989, all countries in Central Europe saw a fall in carbon emissions...Poland is a country in Central Europe.

回答它至少需要：

- 词、短语、句子的含义；
- 跨句推理：把“all countries in Central Europe”实例化到 Poland；
- 把“fall in emissions”等价转换为“reduce emissions”。

这是句子层语义 (sentential semantics) 的任务，而不是单词 WSD 能解决的。

15.1.3 含义的两种定义

形式语义学 (Formal Semantics) 给出两个互补定义：

- **Montague 立场**：自然语言和逻辑学家的人工语言没有本质差异，可在统一数学框架里描述其句法和语义。
- **Wittgenstein 立场**：知道一句话的含义 = 知道世界要满足什么条件才使它为真。即“含义 = truth conditions”。

[EX] 例题 15.1 (真值条件)

Every student works 真值条件：当且仅当 student 集合 $\subseteq$ work 集合。对应 FOL: $\forall x. \text{stud}'(x) \rightarrow \text{work}'(x)$ 。

15.2 含义表示语言 (MRL) 的五条要求

要求	解释	反例
Compositional 组合性	整句含义 = 各部分含义 + 组合规则的函数	把整句直接 hash 为一个原子 token
Verifiable 可验证	能据 MR + 知识库判断句子真假	含义是一个无法对齐 KB 的自然语言改写
Unambiguous 无歧义	同一 MR 对应唯一解释；歧义句应有多多个 MR	time flies like an arrow 仅给一个 MR
Canonical 规范化	同义句应有同一 MR	Beihua serves Sichuan hotpot 与 Spicy hotpot dishes are served by Beihua 不同 MR
Inference + Expressivity 可推理且表达力强	既能直接验证，也能由 MR + KB 演绎答案；同时覆盖足够语言现象	仅支持原子事实，不支持量词、否定

[WARN] 易错点

Vagueness $\neq$ Ambiguity。I want to eat Italian food 中“Italian food”具体指什么是 vague (模糊但单一含义)，不应该派生多个 MR；而 time flies like an arrow 才是 ambiguity，需要多个 MR。

15.3 一阶逻辑 (First-Order Logic, FOL)

一阶逻辑是同时满足以上要求的最广为接受的 MRL。

定义 15.1 (FOL 词汇表)

FOL 表达式由 **terms** 组合而成：

- **常量 (constant)**：唯一指向某实体；Beijing, China, 2021。一个实体可由多个常量指代 (The Evening Star $\equiv$ Venus)。
- **变量 (variable)**： x, y, z 等，遍历所有实体。
- **函数符号 (function)**：间接指实体，如 president(EU), father(John)。句法上像一元 predicate，但返回的是实体而非真值。
- **谓词符号 (predicate)**： n -元 predicate 表示 n 元组上的关系；nation(China), member-of(UK, EU), introduced(John, Marie, Sue)，其元数记作 predicate/ N 。

复杂表达式用**逻辑联结词** $\neg, \vee, \wedge, \Rightarrow$ 与**量词** $\forall, \exists$ 构造。

定义 15.2 (谓词的语义)

一个元数为 N 的谓词 P/N 表示满足它的 N 元组集合:

$$\llbracket P/N \rrbracket = \{(x_1, \dots, x_N) \in U^N : P(x_1, \dots, x_N) \text{ 在世界中为真}\}. \quad (15.1)$$

若所有参数都被实例化为常量, predicate 表达式有真值 (true/false), 由它是否落在 $\llbracket P/N \rrbracket$ 决定。

15.3.1 量词

量词	形式	直觉
$\forall$ universal	$\forall x. P(x) \Rightarrow Q(x)$	“所有 P 都 Q ”, 与所有 P 实例的合取等价
$\exists$ existential	$\exists x. P(x) \wedge Q(x)$	“存在某个 P 同时 Q ”, 与所有 P 实例的析取等价

$$\forall x. \text{cat}(x) \Rightarrow \text{mammal}(x). \quad (15.2)$$

$$\exists x. \text{cat}(x) \wedge \text{owns}(\text{Marie}, x). \quad (15.3)$$

[WARN] 易错点: $\exists$ 配 $\wedge$, $\forall$ 配 $\Rightarrow$

写 Marie owns a cat 不要写成 $\exists x. \text{cat}(x) \Rightarrow \text{owns}(\text{Marie}, x)$ 。因为 $A \Rightarrow B$ 在 A 为假时也为真——只要世界上存在任何非猫实体, 原命题就被空虚地 (vacuously) 满足, 与“Marie 真的有一只猫”无关。反之 $\forall x. \text{cat}(x) \wedge \text{mammal}(x)$ 也是错的: 它要求所有实体既是猫又是哺乳动物。

15.3.2 联结词

$\neg, \wedge, \vee, \Rightarrow$ 与命题逻辑相同。带变量的 predicate 在被量词约束前没有真值——

$$\text{likes}(x, \text{Kim}) \text{ 是集合 } \{x : x \text{ 喜欢 Kim}\}, \quad (15.4)$$

$$\forall x. \text{likes}(x, \text{Kim}) \text{ 才有真值}. \quad (15.5)$$

15.4 事件 Reification 与 Entailment

15.4.1 把事件提升为实体

简单 predicate 难以表达“什么时候、在哪里、和谁一起”。Davidsonian 思路把事件本身作为变量。

[EX] 例题 15.2 (事件 reification)

John gave Mary a book:

$$\exists e, z. \text{Giving}(e) \wedge \text{Giver}(e, \text{John}) \wedge \text{Giver}(e, \text{Mary}) \wedge \text{Given}(e, z) \wedge \text{Book}(z). \quad (15.6)$$

John gave Mary a book on Tuesday:

$$\exists e, z. \text{Giving}(e) \wedge \text{Giver}(e, \text{John}) \wedge \text{Giver}(e, \text{Mary}) \wedge \text{Given}(e, z) \wedge \text{Book}(z) \wedge \text{Time}(e, \text{Tuesday}). \quad (15.7)$$

新的角色 (time, place, manner, instrument...) 只需新加一个 binary predicate, 不改变原有结构——这就是 reification 的核心好处。

15.4.2 Entailment

A entails B ($A \Rightarrow B$) 表示在所有满足 A 的世界中 B 也成立。事件 reification 让 entailment 几乎免费：

公式 (15.7) $\Rightarrow$ 公式 (15.6)，因为前者只是后者多加一个合取项，逻辑上更强。这正对应自然语言中“John gave Mary a book on Tuesday entails John gave Mary a book”。

[NOTE] 两条推理路径

- **KB-based inference**: 把 NL 转 KB 表达式，按 KB 做形式推理 $\rightarrow$ 精确、可验证，但局限于建模过的领域。
- **Language-based inference (NLI)**: 直接判断 utterance A 是否蕴含 utterance B $\rightarrow$ 覆盖广，但主观、近似。现代系统经常两者结合：LLM 给候选 + 形式化系统验证。

15.5 λ 演算：让 FOL 可以组合

FOL 本身不直接说明“如何把词级含义拼成句子级含义”。引入 λ 表达式即可。

定义 15.3 (λ 表达式)

$$\lambda x. \varphi(x) \quad (15.8)$$

是一个未命名函数，把 entity (变量 x) 映射到 FOL 表达式 $\varphi(x)$ 。

定义 15.4 (β -reduction)

将 λ 表达式应用到一个 term 时，用该 term 替换形式参数并去掉 λ ：

$$(\lambda x. \text{run}(x))(\text{Jane}) \rightsquigarrow \text{run}(\text{Jane}). \quad (15.9)$$

嵌套 λ 允许多元函数：

$$(\lambda y. \lambda x. \text{love}(x, y))(\text{pizza}) \rightsquigarrow \lambda x. \text{love}(x, \text{pizza}). \quad (15.10)$$

再代入 Marie：

$$(\lambda x. \text{love}(x, \text{pizza}))(\text{Marie}) \rightsquigarrow \text{love}(\text{Marie}, \text{pizza}). \quad (15.11)$$

[WARN] 易错点

λ 演算中变量名要重命名以避免捕获：如果替换的 term 含同名自由变量，需要先 α -rename。考试一般不深究，但写嵌套表达式要小心。

15.5.1 λ -DCS：工程化的简化形式

Lambda Dependency-based Compositional Semantics (Liang, 2011)

λ -DCS 把 λ 表达式画成 join/intersection 图，便于神经/统计 parser 学习。GeoQuery 之类数据库 QA 系统广泛使用 (Berant et al., 2013)。

15.6 用 CFG 做组合性语义分析

15.6.1 总思路

句法解析 + 词汇语义 $\Rightarrow$ 句子含义

给每条 CFG 规则附上一个语义附加 (semantic attachment):

$$A \rightarrow \alpha_1 \cdots \alpha_n \quad \{ A.\text{sem} = f(\alpha_j.\text{sem}, \dots, \alpha_k.\text{sem}) \}. \quad (15.12)$$

一元规则常常直接 copy; 二元规则常常 λ -apply。

算法 15.1 (Syntax-Driven Semantic Analysis, pipeline 版)

输入: 句子 s , 带 semantic attachment 的 CFG G 。输出: 句子的 MR $S.\text{sem}$ 。

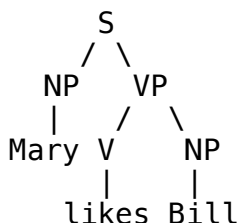
伪代码:

1. 用 CKY / Earley 对 s 在 G 下做句法解析, 得到分析树 T
2. 对 T 做后序遍历 (post-order):
 - 若节点是叶子 (词): 从词典查 lex.sem
 - 若节点是内部规则 $A \rightarrow \alpha_1 \dots \alpha_n$:
按附加函数 f 计算 $A.\text{sem}$, 做 β -reduction
3. 返回根节点 $S.\text{sem}$

复杂度: 句法 CKY 为 $O(n^3|G|)$, semantic attachment 每节点常数时间 (假设 λ 表达式深度有界), 因此总体仍是 $O(n^3|G|)$ 。

15.6.2 玩具例: Mary likes Bill

句法树:



词汇语义:

$$\text{Mary} \mapsto \text{mary}', \quad \text{Bill} \mapsto \text{bill}', \quad \text{likes} \mapsto \lambda y. \lambda x. \text{like}'(x, y). \quad (15.13)$$

自底向上:

$$\text{NP.sem} = \text{mary}', \quad \text{NP.sem} = \text{bill}', \quad (15.14)$$

$$\text{VP.sem} = \text{V.sem}(\text{NP.sem}) = \lambda x. \text{like}'(x, \text{bill}'), \quad (15.15)$$

$$\text{S.sem} = \text{VP.sem}(\text{NP}_{\text{subj}}.\text{sem}) = \text{like}'(\text{mary}', \text{bill}'). \quad (15.16)$$

15.6.3 不那么玩具的例子: AyCaramba serves Meat

词典:

ProperNoun	-> AyCaramba	{ AyCaramba }
MassNoun	-> meat	{ Meat }
NP	-> ProperNoun	{ ProperNoun.sem }
NP	-> MassNoun	{ MassNoun.sem }
Verb	-> serves	{ $\lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y)$ }
VP	-> Verb NP	{ Verb.sem(NP.sem) }
S	-> NP VP	{ VP.sem(NP.sem) }

[EX] 例题 15.3 (worked example)

Step 1 —VP:

$$\begin{aligned} \text{VP.sem} &= (\lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y))(\text{Meat}) \\ &= \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, \text{Meat}). \end{aligned} \quad (15.17)$$

Step 2 —S:

$$\text{S.sem} = \text{VP.sem}(\text{AyCaramba}) = \exists e. \text{Serving}(e) \wedge \text{Server}(e, \text{AyCaramba}) \wedge \text{Served}(e, \text{Meat}). \quad (15.18)$$

15.6.4 麻烦来了：Every kid cries

目标 MR:

$$\forall x. \text{Kid}(x) \Rightarrow \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x). \quad (15.19)$$

如果套用 $S \rightarrow NP \ VP \ \{ \text{VP.sem}(\text{NP.sem}) \}$ ，且 VP 的 sem 为 $\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)$ ，NP 的 sem 为 $\forall x. \text{Kid}(x) \Rightarrow Q(x)$ ——但这表达式带未约束 Q ，无法直接当作 y 喂进 predicate。会得到非法的 FOL:

$$\exists e. \text{Crying}(e) \wedge \text{Crier}(e, \forall x. \text{Kid}(x) \Rightarrow Q(x)) \quad (\times) \quad (15.20)$$

predicate 的参数必须是 term，不能塞入复杂量化公式。

15.6.5 类型提升 (type raising)

修补思路：让 NP 的 sem 反过来“消费”VP。把 NP 变成一个高阶 functor。

新词典 (含 type raising):

Noun	-> kid	{ $\lambda x. \text{Kid}(x)$ }	
Det	-> Every	{ $\lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x)$ }	
NP	-> Det Noun	{ Det.sem(Noun.sem) }	
ProperNoun	-> Kate	{ $\lambda P. P(\text{Kate})$ }	← type-raised
Verb	-> cries	{ $\lambda x. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x)$ }	
S	-> NP VP	{ NP.sem(VP.sem) }	← 注意顺序反转!

[EX] 例题 15.4 (Every kid cries 全推导)

NP 自底向上:

$$\begin{aligned} \text{NP.sem} &= (\lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x))(\lambda x. \text{Kid}(x)) \\ &= \lambda Q. \forall x. (\lambda x. \text{Kid}(x))(x) \Rightarrow Q(x) = \lambda Q. \forall x. \text{Kid}(x) \Rightarrow Q(x). \end{aligned} \quad (15.21)$$

S:

$$\begin{aligned} \text{S.sem} &= \text{NP.sem}(\text{VP.sem}) = (\lambda Q. \forall x. \text{Kid}(x) \Rightarrow Q(x))(\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)) \\ &= \forall x. \text{Kid}(x) \Rightarrow (\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y))(x) \\ &= \forall x. \text{Kid}(x) \Rightarrow \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x). \quad \checkmark \end{aligned} \quad (15.22)$$

[EX] 例题 15.5 (type raising 对 proper noun)

Kate cries. ProperNoun $\rightarrow$ Kate { $\lambda P. P(\text{Kate})$ }, Verb $\rightarrow$ cries { $\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)$ }.

$$\begin{aligned} \text{S.sem} &= (\lambda P. P(\text{Kate}))(\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y)) \\ &= (\lambda y. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, y))(\text{Kate}) = \exists e. \text{Crying}(e) \wedge \text{Crier}(e, \text{Kate}). \end{aligned} \quad (15.23)$$

15.6.6 完整玩具语法 (考点)

S $\rightarrow$ NP VP	{ NP.sem(VP.sem) }
VP $\rightarrow$ Verb	{ Verb.sem }
VP $\rightarrow$ Verb NP	{ Verb.sem(NP.sem) }
NP $\rightarrow$ Det Noun	{ Det.sem(Noun.sem) }
NP $\rightarrow$ ProperNoun	{ ProperNoun.sem }
Det $\rightarrow$ Every	{ $\lambda P. \lambda Q. \forall x. P(x) \Rightarrow Q(x)$ }
Noun $\rightarrow$ Kid	{ $\lambda x. \text{Kid}(x)$ }
ProperNoun $\rightarrow$ Kate	{ $\lambda P. P(\text{Kate})$ }
Verb $\rightarrow$ cries	{ $\lambda x. \exists e. \text{Crying}(e) \wedge \text{Crier}(e, x)$ }
Verb $\rightarrow$ serves	{ $\lambda y. \lambda x. \exists e. \text{Serving}(e) \wedge \text{Server}(e, x) \wedge \text{Served}(e, y)$ }

[WARN] 易错点: S 规则的方向

同一份语法里 S 既可写成 NP.sem(VP.sem) 也可写成 VP.sem(NP.sem), 但两者不能同时混用: 要么全部 type-raise NP, 要么 NP 直接作 term + VP 作 functor。课件统一采用 type-raised NP 版本。

15.7 Quantifier Scope 与组合性的挑战

[NOTE] 真正的“语义不易”

Every student loves some teacher - $\forall x. \text{Stu}(x) \Rightarrow \exists y. \text{Tea}(y) \wedge \text{love}(x, y)$ (每个学生各爱一位) - $\exists y. \text{Tea}(y) \wedge \forall x. \text{Stu}(x) \Rightarrow \text{love}(x, y)$ (有一位老师, 被所有学生爱) 同一句法树会有不同 quantifier scope $\rightarrow$ 实际系统会显式存储一个 scope-underspecified 中间结构, 再由后处理消歧。

其他难点:

- **Pronoun**: it barked 中 it 提供变量 x 并加 PRON(x); it rained 中 it 是 pleonastic, 不进入语义。
- **Non-compositional MWE**: red tape = bureaucracy, 不能 λ 算出来, 需要词典直接给整体含义。

- **Paraphrase**: Kim seems to sleep 与 It seems that Kim sleeps 句法不同但真值条件相同 → canonical form 要求它们殊途同归。

15.8 Semantic Parsing 的两种实现路线

15.8.1 Integrated (边解析边附加)

在 PCFG / 自底向上 chart parser 的每一次" 完成一个 constituent" 后立即调用 attachment。

- 优点：效率高；语义不合法的子树可以早期剪枝，避免 error propagation。
- 缺点：实现复杂，需要把句法解析与 λ 演算耦合；调试困难。

15.8.2 Pipeline (先句法再语义)

1. 用纯句法 parser 得到 best tree T
2. 后序遍历 T 计算 sem

- 优点：模块化；可以替换为 CCG、依存或神经 parser。
- 缺点：句法错误直接传到语义；遇到 quantifier scope、非组合短语需要大量 hand-crafted 例外。

[WARN] 易错点

“integrate”不是“和神经网络融合”，而是把语义 attachment 嵌入到 CFG 的 chart 构造里。

15.9 学习一个语义解析器

完全人工写词典 +attachment 在真实数据上不可扩展。课程给出的现代化路线：

路线	代表工作	输入/输出
概率 CCG 学习	Zettlemoyer & Collins, 2005; Kwiatkowski et al., 2010	(sent, logical form) 监督学习 lexicon + 概率
Neural seq2seq 语义解析	Dong & Lapata, 2016 (Language to Logical Form with Neural Attention)	直接 seq2seq 生成 logical form
Graph-based MR	AMR (Banarescu et al., 2013)	句子 → 抽象含义图 (DAG)
Text-to-SQL	Spider 2.0 (Lei et al., 2025)	句子 + schema → SQL; LLM 时代主流落地形式
Text-to-SPARQL	用于 Wikidata/Freebase	句子 → SPARQL, 多见于 KGQA
LLM as semantic parser	GPT-style few-shot / instruction-tuned	通过 prompt 直接生成 MR

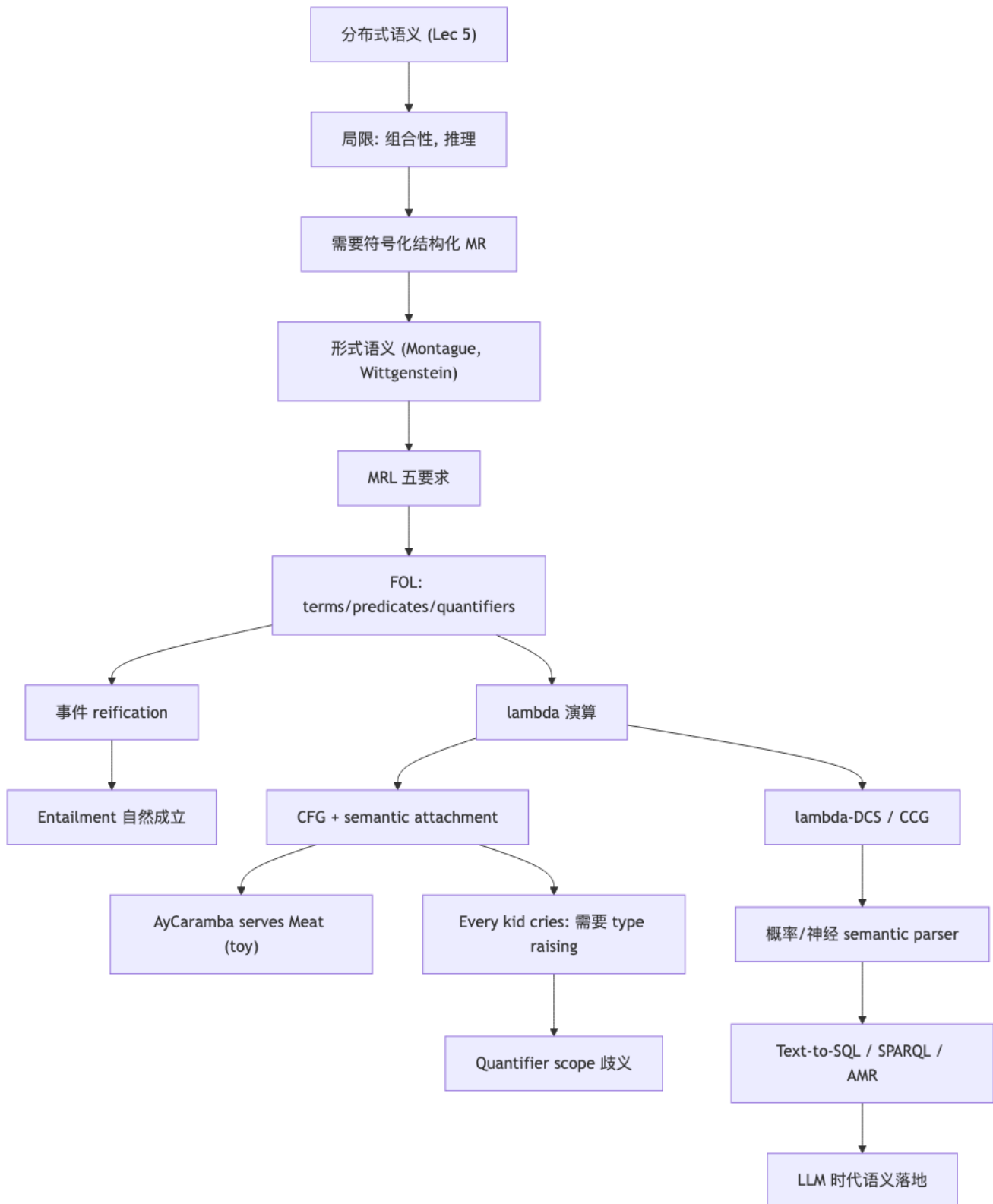
对应世界模型 (world model):

MR 风格	世界模型
FOL / Datalog	形式知识库 (GeoQuery 800 facts)
SPARQL	RDF 图谱 (Wikidata, billions of facts)
SQL	关系数据库表
AMR	节点为概念、边为语义角色的图

[NOTE] 现代视角

课程末尾的反问：“LLM 是否真正理解 today’s semantics?”答案是 LLM 在很多任务上表现良好，但 verifiable / canonical / unambiguous 这些性质并非显式保证；当我们追求**可证明正确的回答**时仍然要回到 formal MR + verification 这条路。

15.10 概念关系



15.11 速查

符号与术语

符号 / 术语	含义
MR	Meaning Representation 含义表示
MRL	Meaning Representation Language 含义表示语言
FOL	First-Order Logic 一阶逻辑
P/N	N -元 predicate
$\lambda x.\varphi$	把实体 x 映到公式 φ 的未命名函数
β -reduction	$(\lambda x.\varphi)(t) \rightsquigarrow \varphi[x \mapsto t]$
reification	把事件提升为可量化变量 e
type raising	把 argument a 包成 $\lambda P.P(a)$, 使其能反过来吃 functor
AMR	Abstract Meaning Representation, 根 DAG
λ -DCS	Lambda Dependency-based Compositional Semantics
canonical form	同义不同表达 $\rightarrow$ 相同 MR

Quantifier 与连接词搭配

句型	MR 模板
Every A is B	$\forall x. A(x) \Rightarrow B(x)$
Some/There exists A that B	$\exists x. A(x) \wedge B(x)$
No A is B	$\forall x. A(x) \Rightarrow \neg B(x)$ 或 $\neg \exists x. A(x) \wedge B(x)$
At most one A is B	$\forall x, y. A(x) \wedge B(x) \wedge A(y) \wedge B(y) \Rightarrow x = y$

CFG 语义附加的两种二元规则

句法	语义
$S \rightarrow NP VP$ (普通)	$VP.sem(NP.sem)$ (NP 是 term)
$S \rightarrow NP VP$ (type-raised)	$NP.sem(VP.sem)$ (NP 是高阶 functor)

[TIP] 期末复习要点

1. 默写 MRL 五条要求；能给出违反每条要求的反例。2. 用 FOL 写出常见量化句子，**确认 $\exists$ 配 $\wedge$ 、 $\forall$ 配 $\Rightarrow$** 。3. 解释为什么需要 event reification，能据此说明 entailment 自然成立。4. 默写 β -reduction 规则；手算 $\lambda y.\lambda x.love(x,y)(pizza)(Marie)$ 。5. 手算 AyCaramba serves Meat 与 Every kid cries 的 S.sem，并解释为何 Every kid cries 必须 type-raise NP。6. 比较 integrate vs pipeline 两种 semantic parsing 实现的优缺点。7. 解释 quantifier scope 歧义 (every student loves some teacher)，并指出它无法被基本 CFG attachment 唯一确定。8. 列举至少 3 种现代 MR / 语义解析形式 (FOL, AMR, SQL, SPARQL, λ -DCS, neural seq2seq)。9. 回答：LLM 是否“理解”这节课定义的语义？给出 verifiable / canonical 角度的反驳。

第十六章 信息检索：从布尔模型到稠密检索 (Lec 10)

[TIP] 学习指南

本章核心：把“信息需求 vs. 文档集合”形式化为一个 ranking 问题；按时间线学习 Boolean $\rightarrow$ Vector Space (TF-IDF) $\rightarrow$ Probabilistic (BM25) $\rightarrow$ Neural (DPR) 四种检索模型，并理解它们对相关性 $f(q, d)$ 的不同建模。前置：第 5 章分布式语义提供 vector space 与相似度；第 4 章 n-gram / 语言模型与统计语言模型直接相关；第 9 章 (Lec 9) 的语义解析给“用 MR 做 retrieval”的另一条路；本讲也是面向最终大作业 (小北助手 IR 组件) 的重要章节。复习关注点：1. IR 的两个阶段 (Retrieval + Ranking) 与 $\tilde{R}(d) \approx R(d)$ 这一近似目标；2. 倒排索引 (inverted index)；3. TF / IDF / TF-IDF 与四种相似度 (Euclidean / Cosine / Dice / Jaccard)；4. BM25 的公式、 k, b 的作用、为什么常胜稠密模型；5. DPR (Dense Passage Retrieval) 的双塔结构、in-batch negatives 与 hard negative；6. 评测、PRF (pseudo relevance feedback) 直觉。

16.1 IR 的任务定义

Information Retrieval (IR)：根据用户信息需求，从大规模数据中检索出相关项。

输入输出三件套：

- 查询 q (token 序列 $w_{q1}, w_{q2}, \dots$)
- 文档集合 $D = \{d_1, d_2, \dots, d_K\}$
- 检索系统：返回 $\tilde{R}(q) \subseteq D$ ，尽可能逼近真实相关集合 $R(q)$ 。

16.1.1 绝对相关 vs 相对相关

两种近似目标：

1. **Solution 1 (绝对相关)**：

$$\tilde{R}(q) = \{d \in D \mid f(d, q) = 1\}. \quad (16.1)$$

“是否相关”是二元判定，难以稳定标注。

2. **Solution 2 (相对相关, ranking)**：

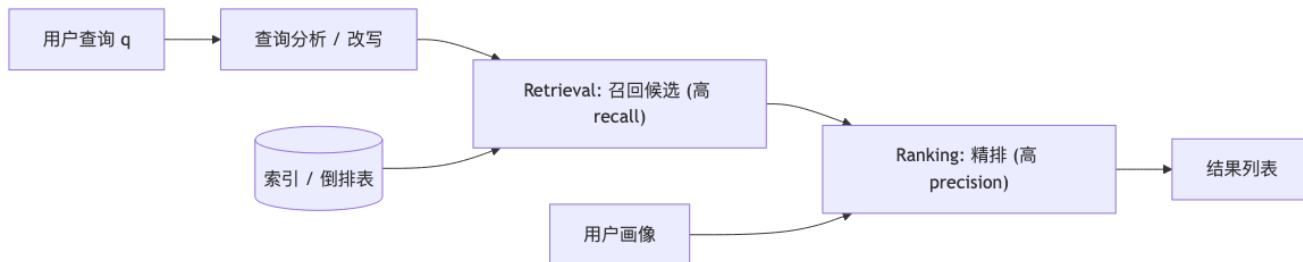
$$\tilde{R}(q) = \{d \in D \mid f(d, q) > \theta\}, \quad (16.2)$$

并保证排序条件

$$f(q, d_i) > f(q, d_j) \iff p(\text{Rel} \mid q, d_i) > p(\text{Rel} \mid q, d_j). \quad (16.3)$$

现代搜索引擎/推荐系统都走 Solution 2。

16.1.2 Pipeline: Retrieval + Ranking



- **Retrieval (召回)**: 宁滥勿缺, 先扩大候选集合。代价低、速度快。
- **Ranking (精排)**: 在候选上做精细打分, 可调用神经/特征工程模型。

[NOTE] 视角统一

这门课把 QA、推荐、广告、社交流推荐、情报分析都看作 IR: 核心都在算 query $\rightarrow$ candidate 的相关性。后续章节的 Question Answering 把 retrieval 当作子模块 (open-domain QA = retriever + reader)。

16.1.3 两个核心子问题

1. 相关性 $f(q, d)$ 怎么建模: 相似度? 概率? 神经评分?
2. q 与 d 怎么表示: bag-of-words? 加权向量? 稠密 embedding?

[WARN] 易错点

不要把 IR 等同于“网页搜索”——课程定义涵盖了 cross-modal 检索 (文本 $\rightarrow$ 图片、用户 $\rightarrow$ 商品)。“ q 必须是关键词”是常见误解, 实际可以是问句、图像、用户行为序列。

16.2 倒排索引 (Inverted Index)

不论用哪种相关性函数, 文档侧都需要一个高效检索结构。

定义 16.1 (倒排索引)

对词表 $V = \{w_1, \dots, w_N\}$, 倒排索引是一个映射

$$\text{posting} : w \mapsto [(d_{i_1}, \text{pos}_1), (d_{i_2}, \text{pos}_2), \dots], \quad (16.4)$$

记录每个词出现在哪些文档以及位置 / 频次。

算法 16.1 (建索引)

输入: 文档集合 D 。输出: 倒排表。

伪代码:

```
for each document d in D:
    tokens = tokenize(d)
    for each (position, w) in enumerate(tokens):
        posting[w].append((doc_id(d), position))
sort each posting list by doc_id
```

复杂度：建索引 $O(\sum_d |d|)$ ；查询时取若干 posting list 做交/并 $O(\sum |\text{posting}|)$ 。

[EX] 例题 16.1 (倒排索引)

文档：
d1: I like machine learning
d2: Machine learning is different with deep learning
d3: Information retrieval is important for many applications
d4: I like information retrieval
d5: Machine learning is useful for ranking in search engine
部分倒排表：
like -> [d1, d4]
machine -> [d1, d2, d5]
learning -> [d1, d2, d2, d5] (d2 出现两次)
information -> [d3, d4]
retrieval -> [d3, d4]
ranking -> [d5]

16.3 Boolean Model

最早的 IR 范式。用 AND/OR/NOT 把查询写成布尔表达式，对倒排表做集合运算。

16.3.1 形式化

查询 $q = \text{“like AND information AND retrieval”}$ 等价于：

$$\tilde{R}(q) = \text{posting}(\text{like}) \cap \text{posting}(\text{information}) \cap \text{posting}(\text{retrieval}). \tag{16.5}$$

[EX] 例题 16.2 (Boolean retrieval)

继续例题 16.1：- $\text{posting}(\text{like}) = \{d1, d4\}$ - $\text{posting}(\text{information}) = \{d3, d4\}$ - $\text{posting}(\text{retrieval}) = \{d3, d4\}$
- $\text{AND-result} = \{d4\}$
若查询是 like AND NOT learning：- $\text{posting}(\text{like}) = \{d1, d4\}$ - $\text{posting}(\text{learning}) = \{d1, d2, d5\}$ - $\{d1, d4\} \setminus \{d1, d2, d5\} = \{d4\}$

16.3.2 优缺点

优点	缺点
精确、快 (集合运算)	用户难写布尔表达式
易于解释	不区分词的重要性 (词都被 0/1 处理)
与倒排表天然契合	无法 ranking, 只能给”命中 / 未命中”
	不支持近似匹配、同义词

[WARN] 易错点

“Boolean Model 慢”的说法有误：倒排表的交并是最快的检索方式之一。它的真正缺点是用户体验差与无法排序。

16.4 Vector Space Model (VSM)

16.4.1 总思路

把 q 与 d 都嵌入 $|V|$ 维向量空间，用向量距离 / 角度刻画相关性。

“We did talk about this before”——第 5 章 distributional semantics 把”语义”嵌入向量；IR 把”相关性”嵌入同一类向量空间。

16.4.2 Bag-of-Words 表示

把文档表示为词频向量

$$\mathbf{d} = (n_{w_1,d}, n_{w_2,d}, \dots, n_{w_{|V|},d}), \quad (16.6)$$

忽略词序。简单、稀疏、与倒排索引完全兼容。

16.4.3 TF / IDF / TF-IDF

Term Frequency (TF) 衡量词在文档中的局部重要性。课件给出的两种常用形式：

$$\text{TF}_{t,d} = \text{count}(t, d), \quad (16.7)$$

或 log-scaled：

$$\text{TF}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d), & \text{count}(t, d) > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (16.8)$$

log-scaled 解决了一个词出现 100 次未必比出现 10 次重要 10 倍的问题。

Inverse Document Frequency (IDF) 衡量词的全局区分度：

$$\text{IDF}_t = \log_{10} \frac{N}{\text{DF}_t}, \quad (16.9)$$

其中 N 是文档总数， DF_t 是包含词 t 的文档数。IDF 越大，词越稀有、越有区分力 (e.g. quantum vs the)。

TF-IDF：

$$\text{TF-IDF}(t, d) = \text{TF}_{t,d} \times \text{IDF}_t. \quad (16.10)$$

[EX] 例题 16.3 (TF-IDF 手算)

假设 $N = 5$ ，文档同例题 16.1。- “learning”出现于 d1, d2, d5: $\text{DF} = 3$, $\text{IDF} = \log_{10}(5/3) \approx 0.222$ 。
- “retrieval”出现于 d3, d4: $\text{DF} = 2$, $\text{IDF} = \log_{10}(5/2) \approx 0.398$ 。- “I”出现于 d1, d4: $\text{DF} = 2$, $\text{IDF} \approx 0.398$ ，但作为停用词通常被去掉。
在 d4 (“I like information retrieval”) 里：- $\text{TF}(\text{retrieval}, d4) = 1$, $\text{TF-IDF} \approx 1 \times 0.398 = 0.398$ ；- $\text{TF}(\text{learning}, d4) = 0$, $\text{TF-IDF} = 0$ 。

16.4.4 相似度函数

设 $q, d \in \mathbb{R}^{|V|}$ (一般用 TF-IDF 权重)。课件给出四种 $f(q, d)$:

Euclidean 距离 (距离越小越相关):

$$f_{\text{Euc}}(q, d) = \frac{1}{\sqrt{\sum_i (d_i - q_i)^2}}. \quad (16.11)$$

[WARN] 易错点

Euclidean 对**长度敏感**: 一篇长文档里所有词频都更大, 欧氏距离会被拉大, 相关性变小。IR 几乎不直接用它。

Cosine 相似度 (最常用):

$$f_{\text{cos}}(q, d) = \cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}. \quad (16.12)$$

只看方向, 自动消除长度差异。

Dice 系数:

$$f_{\text{Dice}}(q, d) = \frac{2 q \cdot d}{\|q\|^2 + \|d\|^2}. \quad (16.13)$$

Jaccard 系数:

$$f_{\text{Jac}}(q, d) = \frac{q \cdot d}{\|q\|^2 + \|d\|^2 - q \cdot d}. \quad (16.14)$$

[EX] 例题 16.4 (cosine 手算)

设词表 $V = \{\text{information, retrieval, learning}\}$ 。- $q = \text{"information retrieval"} \rightarrow q = (1, 1, 0)$ (或 TF-IDF 加权后 $(0.398, 0.398, 0)$, 方向不变)。- $d_4 = \text{"I like information retrieval"} \rightarrow$ 含词向量 $(1, 1, 0)$ 。
- $d_1 = \text{"I like machine learning"} \rightarrow (0, 0, 1)$ 。

$\cos(q, d_4) = \frac{1 \cdot 1 + 1 \cdot 1 + 0}{\sqrt{2} \cdot \sqrt{2}} = 1$ 。 $\cos(q, d_1) = 0$ 。

排序: $d_4 > d_3 > d_1$ (与直觉一致)。

16.4.5 VSM 的工程难点

- **稀疏 vector**: $|V|$ 通常 $10^5 - 10^7$, 但每文档只激活几百维。需要稀疏存储与倒排辅助计算。
- **同义词 / 多义词**: bag-of-words 不会把“car”与“automobile”关联。Topic models (LSI, pLSI, LDA) 与 word embedding (word2vec, GloVe) 是常见解决方案。
- **长尾词**: IDF 过大可能 dominate, 工程上常加载断或平滑。

16.5 Probabilistic Model: BM25

Binary Independence Model (Robertson & Jones, 1970s) $\rightarrow$ Okapi BM25 (Robertson, 1994): 至今最强的稀疏 baseline。

16.5.1 概率检索原则 (Probability Ranking Principle)

按 $p(\text{Rel} \mid q, d)$ 倒序输出文档即可获得最优性能。但直接建模这个条件概率很难。Binary Independence Model 假设各词独立、且只考虑词出现与否，得到一种 likelihood ratio 形式。

16.5.2 BM25 公式

$$\text{BM25}(q, d) = \sum_{w \in q} \log\left(\frac{N}{\text{df}_w}\right) \cdot \frac{\text{tf}_{w,d}}{k \left(1 - b + b \frac{|d|}{\text{avgdl}}\right) + \text{tf}_{w,d}}. \quad (16.15)$$

各符号：

符号	含义
N	文档总数
df_w	含词 w 的文档数
$\text{tf}_{w,d}$	词 w 在 d 中出现次数
$ d $	文档长度
avgdl	集合中平均文档长度
k	TF 饱和参数，常 $k \in [1.2, 2.0]$
b	文档长度归一化系数，常 $b = 0.75$

公式分两部分理解：

- IDF 项 $\log(N/\text{df}_w)$ ：和 VSM 的 IDF 一致，给稀有词更大权重。
- 饱和 TF 项 $\frac{\text{tf}}{k(\cdot) + \text{tf}}$ ：随 tf 单调增但有上界 1，避免高频词无限叠加。

16.5.3 k 与 b 的作用

[NOTE] 参数直觉

– k 控制 TF 饱和速度。 $k \rightarrow 0$ 时 TF 项退化为 0/1 (“出现就计 1”); $k \rightarrow \infty$ 时退化为线性 TF。常用 $k \approx 1.5$ 。– b 控制长度归一化强度。 $b = 0$ 时不归一化 (长文档天然占便宜); $b = 1$ 时完全归一化 (按 $|d|/\text{avgdl}$ 缩放)。常用 $b = 0.75$ 在两者间折中。

16.5.4 为什么 BM25 仍然强势

原因	解释
倒排索引天然支持	计算单词分数后加和即可，与 cosine 相同
TF 饱和与长度归一	解决了 TF-IDF 两个最常见 bug
无需训练数据	适合 cold-start、垂域语料
强 baseline	现代 dense retrieval 论文几乎都把 BM25 当对比基线

[EX] 例题 16.5 (BM25 手算)

取 $k = 1.5, b = 0.75, N = 5$ 。文档 $d_3 = \text{“Information retrieval is important for many applications”}$, $|d_3| = 7$ 。假设 $\text{avgdl} = 6$ 。查询 $\text{information retrieval}$ 。

- “information”: $\text{df} = 2, \log(5/2) \approx 0.916$; $\text{tf} = 1$ 。归一长度因子 $K = 1.5(1 - 0.75 + 0.75 \cdot 7/6) = 1.5 \cdot (0.25 + 0.875) = 1.6875$ 。分数项 $\frac{1}{1.6875+1} = 0.372$ 。贡献 $0.916 \times 0.372 = 0.341$ 。
- “retrieval”: 同上贡献 0.341 。
- $\text{BM25}(q, d_3) = 0.341 + 0.341 = 0.682$ 。

对 d_5 (“Machine learning is useful for ranking in search engine”, 含 0 个查询词), $\text{BM25} = 0$ 。故排序 $d_3, d_4 > d_1, d_2, d_5$ 。

16.6 Pseudo Relevance Feedback (PRF, 补充)

[NOTE] 直觉

即便没有用户反馈, 假设 top-k 检索结果“基本相关”, 将其高频词回填进查询, 重新检索一次, 常能提升 recall。

Rocchio 算法 (经典):

$$\mathbf{q}_{\text{new}} = \alpha \mathbf{q} + \beta \frac{1}{|D_R|} \sum_{d \in D_R} \mathbf{d} - \gamma \frac{1}{|D_{NR}|} \sum_{d \in D_{NR}} \mathbf{d}, \quad (16.16)$$

其中 D_R 是相关文档集, D_{NR} 是非相关集, $\alpha + \beta + \gamma = 1$ 。PRF 把 top-k 当作 D_R 的近似, 无需真实标注。

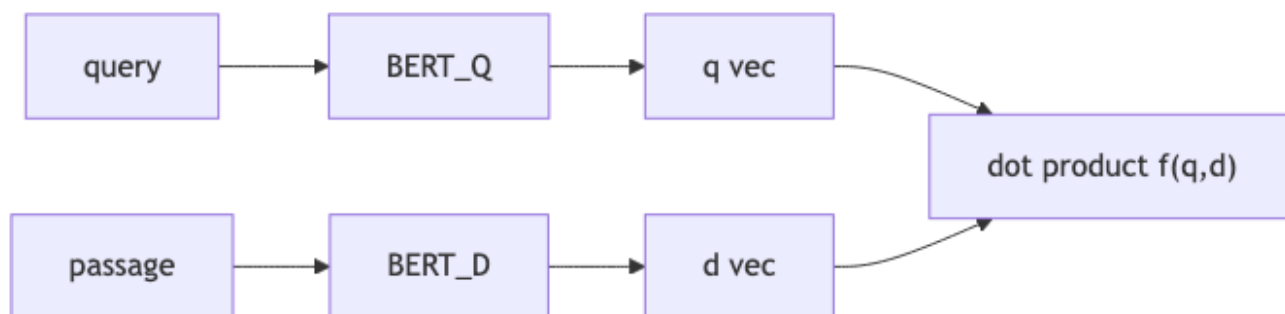
16.7 Neural Model: Dense Passage Retrieval (DPR)

16.7.1 整体思路

把 q 与 d 用神经编码器映到稠密低维向量 $\mathbf{q}, \mathbf{d} \in \mathbb{R}^D$ (通常 $D = 768$):

$$\mathbf{q} = \text{Enc}_Q(q), \quad \mathbf{d} = \text{Enc}_D(d), \quad f(q, d) = \mathbf{q}^\top \mathbf{d}. \quad (16.17)$$

“双塔”/ “biencoder”结构。



文档侧的向量可以离线计算并放入 FAISS / ScaNN 等 ANN 索引; 查询侧只需在线编码一次, 毫秒级返回 top-k。

16.7.2 训练目标

经典 contrastive loss: 对每个 (q, d^+) 正例与 $n - 1$ 个负例 d_i^- ,

$$\mathcal{L} = -\log \frac{\exp(\mathbf{q}^\top \mathbf{d}^+)}{\exp(\mathbf{q}^\top \mathbf{d}^+) + \sum_{i=1}^{n-1} \exp(\mathbf{q}^\top \mathbf{d}_i^-)}. \quad (16.18)$$

负例选择直接决定模型质量:

负例类型	说明
Random negative	随机文档; 太容易, 模型学不到细粒度
BM25-hard negative	BM25 得分高但不包含答案的段落; 含义近、易混淆
Positive of OTHER questions	别人的 positive; 分布接近, 但任务无关
In-batch negative	同一 mini-batch 中其他问题的 positive 段落

16.7.3 In-Batch Negatives

[EX] 例题 16.6 (in-batch negatives)

一个 mini-batch 内有 n 对 (q_i, d_i^+) 。设 $Q \in \mathbb{R}^{d \times n}$ 、 $P \in \mathbb{R}^{d \times n}$ 分别是 query / passage embedding。计算

$$S = Q^\top P \in \mathbb{R}^{n \times n}, \quad (16.19)$$

第 i 行第 i 列是 (q_i, d_i^+) 的相似度 (正例), 其余 $n - 1$ 列是 $(q_i, d_j^+), j \neq i$, 作为负例。整个 batch 总共得到 n^2 对 (query, passage) 但只需要 $2n$ 次编码——计算效率极高。

DPR 论文最强配置: in-batch negatives + 1 个 BM25 hard negative。

16.7.4 DPR 经验教训

- 只用几万对 (Q, A) 数据 + Reading Comprehension 数据足以训练; 不需要 large-scale pretraining。
- 用 RC vs. QA 数据训练差异很小 (41.0 vs 41.5 on Natural Questions)。
- BM25 与 DPR 互补: top-k union (BM25 + DPR) 略优于单独 DPR。
- 难在收集训练数据: 高质量 (q, d^+) 对在垂域里成本高。

[WARN] 易错点

不要把 “in-batch negative” 等同于 “随机负例”。前者负例来自当前 batch 的其他正例段落, 分布更贴近真实, 效果显著好于 “全语料随机采样”。

16.7.5 神经检索的其它架构

名称	结构	特点
Cross-encoder	$[q; d]$ 一起进 BERT, 输出标量	精度高, $O(NK)$, 慢
Biencoder (DPR)	独立编码 + dot product	快, 可离线索引
ColBERT	每 token 留一个 vec, late interaction	折中: 精度高 + 可索引
Sparse neural (SPLADE)	神经预测稀疏 BoW	与倒排索引兼容, 融合 BM25 优点

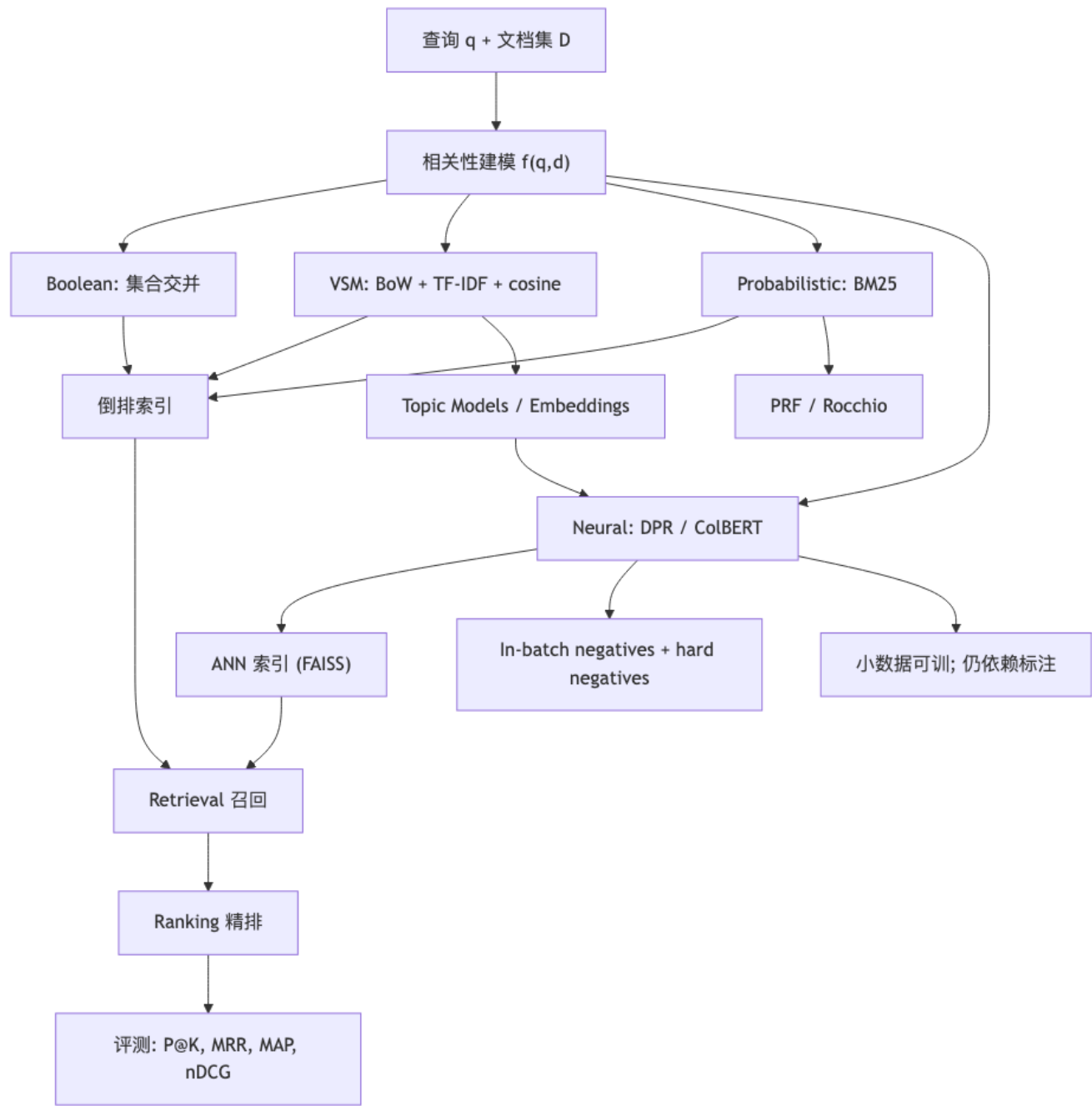
16.8 评测指标 (补充)

指标	直觉
Precision@K	前 K 个里相关比例
Recall@K	真相关被前 K 个覆盖比例
MRR (Mean Reciprocal Rank)	$\frac{1}{Q} \sum_q \frac{1}{\text{rank}_q}$, 关注首个相关文档位置
MAP (Mean Average Precision)	对每个 query 取所有 recall level 的 precision 平均, 再对 query 取平均
nDCG	折扣累积增益, 考虑 graded relevance 与位置衰减

[NOTE] 何时看哪个指标

- 网页搜索: MRR / nDCG@10 (用户只看前几条)。
- QA retrieval: Recall@20 / Recall@100 (后面要喂给 reader)。
- 知识检索: MAP (覆盖所有相关文档)。

16.9 概念关系



16.10 速查

公式总表

模块	公式
TF (log)	$1 + \log_{10} \text{count}(t, d)$
IDF	$\log_{10}(N / \text{DF}_t)$
TF-IDF	$\text{TF}_{t,d} \text{IDF}_t$

模块	公式
Cosine	$\frac{q \cdot d}{\ q\ \ d\ }$
Dice	$\frac{2 q \cdot d}{\ q\ ^2 + \ d\ ^2}$
Jaccard	$\frac{q \cdot d}{\ q\ ^2 + \ d\ ^2 - q \cdot d}$
BM25	$\sum_{w \in q} \log \frac{N}{\text{df}_w} \cdot \frac{\text{tf}_{w,d}}{k(1-b+b d /\text{avgdl}) + \text{tf}_{w,d}}$
DPR	$f(q, d) = \text{Enc}_Q(q)^\top \text{Enc}_D(d)$
In-batch	$S = Q^\top P \in \mathbb{R}^{n \times n}, \text{ 对角为正例}$

模型对照

模型	q 表示	d 表示	$f(q, d)$	是否需训练	优势
Boolean VSM	布尔表达式 TF-IDF 稀疏 vec	词集合 同	集合交并 cosine 等	否 否	精确、快 可排序, 工程简单
BM25	词袋	词袋 + 长度	概率打分	否	仍是最强 稀疏 baseline
DPR	稠密 vec (BERT)	稠密 vec (BERT)	dot product	是 (需要 (Q,P+) 对)	语义匹配, 跨同义词
Cross-encoder	联合输入	联合输入	scalar	是	精度最高, 但慢

Boolean / VSM / BM25 / DPR 易混点

易混	区分
TF-IDF vs BM25	BM25 多了 TF 饱和与长度归一; 公式形式相似但行为差异大
Cosine vs Euclidean	Cosine 对长度鲁棒; Euclidean 不行
In-batch negative vs Random negative	前者来自 batch, 与目标更相关; 后者完全随机
Retrieval vs Ranking	前者 high recall + low cost; 后者 high precision + 可贵
Sparse vs Dense	稀疏可用倒排, 词级匹配; 稠密需 ANN, 语义匹配

[TIP] 期末复习要点

1. 写出 IR 的两步 pipeline, 并解释 retrieval 与 ranking 关注的指标差异。2. 默写倒排索引的数据结构, 举例说明 Boolean retrieval 中 AND/NOT 怎样执行。3. 默写 TF, IDF, TF-IDF; 说明 log-scaled TF 的动机。4. 写出 cosine、Dice、Jaccard, 并解释 cosine 在 IR 中最常用的原因 (长度鲁棒)。5. 默写 **BM25 公式**; 解释 k , b 的作用与常用取值; 能手算一个简单例子。6. 解释为什么 BM25 至今仍是强基线, 并指出它的局限 (语义匹配差、依赖词形)。7. 画 DPR 双塔结构图, 写出 in-batch negative 的矩阵形式 $S = Q^\top P$ 。8. 列举 4 种 negative sampling 策略, 并说明 DPR 论文最强配置。9. 解释 PRF / Rocchio 的更新公式; 说明何时奏效、何时失败 (top-k 噪声大时反而伤害)。10. 能为”小北助手 IR 组件 (面向校内、含留学生)”画出 architecture: query 语种识别 + 翻译 $\rightarrow$ bilingual retrieval (BM25 + 多语 DPR) $\rightarrow$ cross-encoder 精排 $\rightarrow$ 答案生成。